BOUBLIGHA Abderrahman

TP : Prétraitement d’un Dataset en Machine Learning

1- J’ai installé toutes les bibliothèques nécessaires.

```
PS C:\Users\LOQ\Desktop\master\machine_learning\data_cleaning> pip install pandas numpy matplotlib seaborn scikit-learn imbalanced-learn
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pandas in c:\users\loq\appdata\roaming\python\python314\site-packages (3.0.1)
Requirement already satisfied: numpy in c:\users\loq\appdata\roaming\python\python314\site-packages (2.4.3)
Collecting matplotlib
  Downloading matplotlib-3.10.8-cp314-cp314-win_amd64.whl.metadata (52 kB)
Collecting seaborn
  Downloading seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Collecting scikit-learn
  Downloading scikit_learn-1.8.0-cp314-cp314-win_amd64.whl.metadata (11 kB)
Collecting imbalanced-learn
  Downloading imbalanced_learn-0.14.1-py3-none-any.whl.metadata (8.9 kB)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\loq\appdata\roaming\python\python314\site-packages (from pandas) (2.9.0.post0
)
```

2- J’ai importé pandas pour lire les données depuis le dataset (fichier CSV).

1- Charger le datasetdepuis fichier csv

```python
import pandas as pd

data = pd.read_csv("../Airbnb_Open_Data.csv", low_memory = False)
```

✓ 0.8s

3- J’ai utilisé data.head(10) pour afficher les 10 premières lignes du dataframe et data.tail(10) pour afficher les 10 dernières lignes du dataframe.

```python
#afficher les 10 premiere line et les 10 dernieres line du dataframe
data.head(10)
```

✓ 0.0s Python

	id	NAME	host id	host_identity_verified	host name	neighbourhood group	neighbourhood	lat	long	country	...	service fee	minir ni
0	1001254	Clean & quiet apt home by the park	80014485718	unconfirmed	Madaline	Brooklyn	Kensington	40.64749	-73.97237	United States	...	$193	
1	1002102	Skylit Midtown Castle	52335172823	verified	Jenna	Manhattan	Midtown	40.75362	-73.98377	United States	...	$28	

Et data.info() pour voir les noms des colonnes, le nombre de valeurs non nulles, les types de données, etc.

```
[11]  data.info()
      ✓ 0.0s

<class 'pandas.DataFrame'>
RangeIndex: 102599 entries, 0 to 102598
Data columns (total 26 columns):
 #   Column                          Non-Null Count   Dtype
---  ------                          --------------   -----
 0   id                              102599 non-null  int64
 1   NAME                            102349 non-null  str
 2   host id                         102599 non-null  int64
 3   host_identity_verified          102310 non-null  str
 4   host name                       102193 non-null  str
 5   neighbourhood group             102570 non-null  str
 6   neighbourhood                   102583 non-null  str
 7   lat                             102591 non-null  float64
 8   long                            102591 non-null  float64
```

PARTIE 2 : Nettoyage des données

1-J'ai utilisé data.duplicated() pour vérifier les doublons dans le dataset, où **False** signifie que la ligne est unique et **True** qu'elle est dupliquée, puis avec data.duplicated().sum() j'ai trouvé qu'il y a **541 lignes dupliquées**, et enfin data.duplicated().value_counts() m'a permis de voir qu'il y a **102058 lignes sans doublons** et **541 lignes dupliquées**.

```
[153]  data.duplicated()
       ✓ 0.2s                                                    Python

0         False
1         False
2         False
3         False
4         False
          ...
102594     True
102595     True
102596     True
102597     True
102598     True
Length: 102599, dtype: bool

[154]  data.duplicated().sum()
       ✓ 0.2s                                                    Python

np.int64(541)

[155]  data.duplicated().value_counts()
       ✓ 0.2s                                                    Python

False    102058
True        541
Name: count, dtype: int64

Generate   + Code   + Markdown
```

2-J’ai utilisé data.drop_duplicates(inplace=True) pour supprimer les doublons, puis avec data.duplicated().sum() j’ai vérifié qu’il n’y en a plus.

```
data.drop_duplicates(inplace = True)
#inplace true means work on the same dataframe and not return a new one
data.duplicated().sum()
[156] ✓ 0.5s
... np.int64(0)
```

3-J’ai utilisé data.isna().sum() pour voir les valeurs manquantes dans chaque colonne, puis comme la colonne **last review** contient beaucoup de valeurs nulles, je l’ai supprimée avec data.drop(columns=["last review"], inplace=True)

```
#because the last review column has a lot of missing values we will drop it
data.drop(columns = ["last review"], inplace = True)
8] ✓ 0.0s
```

4-J’ai supprimé la colonne **NAME** avec data.drop(columns=["NAME"], inplace=True) parce qu’elle est juste marketing et ne sert pas dans l’analyse

```
data.drop(columns = ["NAME"], inplace = True)
#i deleted because it is just for marketing can't help us in our analysis
[161] ✓ 0.0s

# Check Unique Values for each variable
```

5-J’ai utilisé une boucle pour afficher le nombre de valeurs uniques dans chaque colonne avec data[i].nunique(), ce qui m’a permis de voir que certaines colonnes comme **country**, **country code** et **license** ont une seule valeur donc elles ne sont pas utiles pour l’analyse.

```python
# Check Unique Values for each variable.
for i in data.columns.tolist():
    print(f"Number of unique values in {i} is {data[i].nunique()}")
```
✓ 0.0s

```
Number of unique values in id is 102058
Number of unique values in host id is 102057
Number of unique values in host_identity_verified is 2
Number of unique values in host name is 13190
Number of unique values in neighbourhood group is 7
Number of unique values in neighbourhood is 224
Number of unique values in lat is 21991
Number of unique values in long is 17774
Number of unique values in country is 1
Number of unique values in country code is 1
Number of unique values in instant_bookable is 2
Number of unique values in cancellation_policy is 3
Number of unique values in room type is 4
Number of unique values in Construction year is 20
Number of unique values in price is 1151
Number of unique values in service fee is 231
Number of unique values in minimum nights is 153
Number of unique values in number of reviews is 476
Number of unique values in reviews per month is 1016
Number of unique values in review rate number is 5
Number of unique values in calculated host listings count is 78
Number of unique values in availability 365 is 438
Number of unique values in house_rules is 1976
Number of unique values in license is 1
```

6-J'ai utilisé value_counts() pour voir combien il y a de **verified** et **unconfirmed**, puis mode() pour trouver la valeur la plus fréquente (qui est **unconfirmed**), donc j'ai remplacé les valeurs manquantes par **"unconfirmed"** avec fillna() parce que c'est la valeur la plus logique à utiliser

```python
data["host_identity_verified"].value_counts()
```
✓ 0.0s

```
host_identity_verified
unconfirmed    50944
verified       50825
Name: count, dtype: int64
```

```python
data["host_identity_verified"].mode()
```
✓ 0.0s

```
0    unconfirmed
Name: host_identity_verified, dtype: str
```

```python
#data["host_identity_verified"].fillna(data["host_identity_verified"].mode()[0], inplace = True)
data["host_identity_verified"] = data["host_identity_verified"].fillna("unconfirmed")
```
✓ 0.0s

7-J'ai supprimé ces colonnes parce que d'après le nombre de valeurs uniques : **country** et **country code** ont une seule valeur donc elles n'apportent rien, et les colonnes comme **lat**, **long**, **neighbourhood** et **neighbourhood group** ont beaucoup de valeurs différentes mais elles ne sont pas nécessaires pour mon analyse, tandis que **host name** n'est qu'un identifiant texte qui n'aide pas à faire des analyses utiles.

```
data.drop(columns=["host name","neighbourhood group","neighbourhood","lat","long","country","country code"], inplace = True)
✓ 0.0s
```

8-J'ai utilisé value_counts() pour voir les valeurs de **instant_bookable** (True/False), puis comme il y avait des valeurs manquantes j'ai utilisé mode() pour trouver la valeur la plus fréquente et je les ai remplacées avec fillna() (j'ai utilisé l'affectation directe parce que inplace=True n'a pas fonctionné).

```
data["instant_bookable"].value_counts()
✓ 0.0s

instant_bookable
False    51186
True     50767
Name: count, dtype: int64

# data["instant_bookable"].fillna(data["instant_bookable"].mode()[0], inplace = True) inplace true didn't work i don't know why
data["instant_bookable"] = data["instant_bookable"].fillna(data["instant_bookable"].mode()[0])
✓ 0.0s
```

9-J'ai vérifié les valeurs manquantes avec data.isna().sum(), puis pour la colonne **cancellation_policy** (variable catégorielle) j'ai remplacé les valeurs nulles par la valeur la plus fréquente avec mode(), et pour **Construction year** j'ai rempli les valeurs manquantes avec la moyenne puis j'ai converti la colonne en entier avec astype(int)

```
data.isna().sum()
✓ 0.0s

id                                  0
host id                             0
host_identity_verified              0
instant_bookable                    0
cancellation_policy                76
room type                           0
Construction year                 214
price                             247
service fee                       273
minimum nights                    400
number of reviews                 183
reviews per month               15818
review rate number                319
calculated host listings count    319
availability 365                  448
house_rules                     51842
license                        102056
dtype: int64

#categorical variable
data["cancellation_policy"] = data["cancellation_policy"].fillna(data["cancellation_policy"].mode()[0])
✓ 0.0s

data["Construction year"] = data["Construction year"].fillna(data["Construction year"].mean())
data["Construction year"]= data["Construction year"].astype(int)
data
✓ 0.0s
```

10-J'ai nettoyé la colonne **price** en supprimant les symboles comme $, les virgules et les espaces avec str.replace(), puis je l'ai convertie en type numérique avec astype(float) pour pouvoir l'utiliser dans l'analyse

```
#mean when the destribution is normal and median when the distribution is deferent some so big and some so small
data["price"] = (
    data["price"]
    .str.replace("$", "", regex=False)
    .str.replace(",", "", regex=False)
    .str.replace(" ", "", regex=False)
)
data["price"] = data["price"].astype(float)
data
```

11-J'ai utilisé skew() pour mesurer si les données de **price** sont équilibrées ou non, c'est-à-dire voir si la distribution est normale ou penchée vers des grandes ou petites valeurs, et comme le **skew est proche de 0** ça veut dire que les données sont symétriques (normales), donc j'ai utilisé la **mean** parce qu'elle est adaptée dans ce cas, alors que si le skew était grand (distribution déséquilibrée) on utilise la **median** parce qu'elle n'est pas influencée par les valeurs extrêmes.

```
# Get summary stats for non-null values
stats = data['price'].describe()
skew = data['price'].skew()

print(f"Mean: {stats['mean']}, Median: {stats['50%']}, Skew: {skew}")

# Decision logic
if abs(skew) < 0.5:
    print("Strategy: Mean (Symmetric)")
else:
    print("Strategy: Median (Skewed)")
0]  ✓ 0.0s
```

12-Le même chose pour ce colonne aussi J'ai nettoyé la colonne **service fee** en supprimant les symboles ($, espaces, virgules) puis je l'ai convertie en **float**, ensuite j'ai utilisé describe() et skew() pour analyser la distribution, et comme le **skew est proche de 0** (distribution normale) j'ai choisi la **mean**, donc j'ai rempli les valeurs manquantes avec data["service fee"].mean()

```python
data["price"] = data["price"].fillna(data["price"].mean())
data["service fee"] = (
    data["service fee"]
    .str.replace("$", "", regex=False)
    .str.replace(",", "", regex=False)
    .str.replace(" ", "", regex=False)
)
data["service fee"] = data["service fee"].astype(float)

# Get summary stats for non-null values
stats = data['service fee'].describe()
skew = data['service fee'].skew()

print(f"Mean: {stats['mean']}, Median: {stats['50%']}, Skew: {skew}")

# Decision logic
if abs(skew) < 0.5:
    print("Strategy: Mean (Symmetric)")
else:
    print("Strategy: Median (Skewed)")
```

✓ 0.0s

```
Mean: 125.0392493982414, Median: 125.0, Skew: 0.0017217337489740503
Strategy: Mean (Symmetric)
```

```python
data["service fee"] = data["service fee"].fillna(data["service fee"].mean())
data
```

✓ 0.0s

13-J'ai rempli les valeurs manquantes de **minimum nights** avec la **mean** puis je l'ai convertie en entier avec astype(int), ensuite pour **number of reviews** et **reviews per month** j'ai remplacé les valeurs nulles par **0.0** parce que ça signifie aucune activité et il y a des valeus dans kle tableau one la valeur 0.0, pour **review rate number** j'ai mis **2.5** comme valeur moyenne, et pour **calculated host listings count** j'ai mis **1.0** parce que ça représente au moins une annonce.

```python
data["minimum nights"] = data["minimum nights"].fillna(data["minimum nights"].mean())
#convert to int
data["minimum nights"] = data["minimum nights"].astype(int)
```

✓ 0.0s

```python
# number of reviews because some columnes have 0.0
data["number of reviews"] = data["number of reviews"].fillna(0.0)
data["reviews per month"] = data["reviews per month"].fillna(0.0)
```

✓ 0.0s

```python
data["review rate number"]= data["review rate number"].fillna(2.5)
```

✓ 0.0s

```python
data["calculated host listings count"] = data["calculated host listings count"].fillna(1.0)
```

✓ 0.0s

```python
data.isna().sum()
```

✓ 0.0s

14-J'ai vérifié les valeurs manquantes avec data.isna().sum(), puis j'ai supprimé les colonnes **id** et **host id** parce que ce sont juste des identifiants, **availability 365** et **house_rules** parce qu'elles ne sont pas utiles pour mon analyse, et **license** parce qu'elle contient presque que des valeurs nulles

```python
data.isna().sum()
```

✓ 0.0s

```
id                                  0
host id                             0
host_identity_verified              0
instant_bookable                    0
cancellation_policy                 0
room type                           0
Construction year                   0
price                               0
service fee                         0
minimum nights                      0
number of reviews                   0
reviews per month                   0
review rate number                  0
calculated host listings count      0
availability 365                  448
house_rules                     51842
license                        102056
dtype: int64
```

```python
data.drop(columns = ["id","host id","availability 365","house_rules","license"], inplace = True)
data
```

✓ 0.0s

15-J'ai vérifié avec data.isna().sum() et j'ai constaté qu'il n'y a plus de valeurs manquantes, donc le dataset est maintenant propre et prêt pour l'analyse

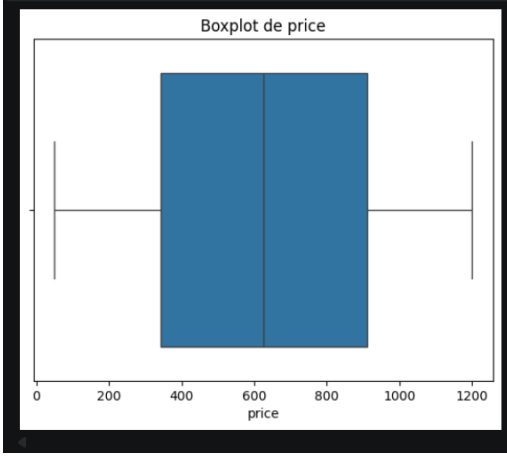
```
data.isna().sum()
✓ 0.0s

host_identity_verified            0
instant_bookable                  0
cancellation_policy               0
room type                         0
Construction year                 0
price                             0
service fee                       0
minimum nights                    0
number of reviews                 0
reviews per month                 0
review rate number                0
calculated host listings count    0
dtype: int64
```

Gestion des valeurs aberrantes

1-J'ai utilisé des boxplots avec sns.boxplot() pour visualiser les valeurs aberrantes (outliers) dans les colonnes price, minimum nights et number of reviews, afin d'identifier les valeurs très élevées ou très faibles par rapport aux autres

```python
columns = ["price", "minimum nights", "number of reviews"]

for col in columns:
    plt.figure()
    sns.boxplot(x=data[col])
    plt.title(f"Boxplot de {col}")
    plt.show()
```

✓ 0.7s



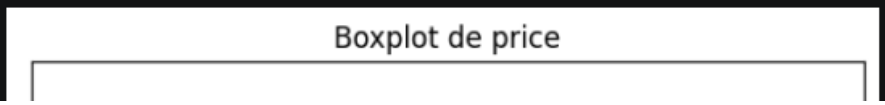
2-J'ai créé une fonction pour supprimer les valeurs aberrantes en utilisant la méthode **IQR** (quartiles), où je calcule **Q1**, **Q3** et **IQR**, puis je garde seulement les valeurs entre **Q1 - 1.5×IQR** et **Q3 + 1.5×IQR**, ensuite j'ai appliqué cette fonction sur **price**, **minimum nights** et **number of reviews**, et enfin j'ai réaffiché les boxplots pour vérifier que les outliers ont été supprimés.

```python
def remove_outliers_all(df, cols):
    for col in cols:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1

        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR

        df = df[(df[col] >= lower) & (df[col] <= upper)]

    return df

data = remove_outliers_all(data, ["price", "minimum nights", "number of reviews"])
```

[2] ✓ 0.0s

```python
columns = ["price", "minimum nights", "number of reviews"]

for col in columns:
    plt.figure()
    sns.boxplot(x=data[col])
    plt.title(f"Boxplot de {col}")
    plt.show()
```

[3] ✓ 0.5s



Partie 3 : Transformation des Données1-J'ai utilisé le One-Hot

Encoding pour transformer les variables catégorielles (host_identity_verified, instant_bookable, cancellation_policy, room type) en valeurs numériques, parce que les modèles de machine learning ne comprennent que les nombres et pas le texte.

Le principe du One-Hot Encoding est de créer une nouvelle colonne pour chaque catégorie (par exemple verified et unconfirmed) et de mettre 1 si la ligne appartient à cette catégorie et 0 sinon.

J'ai utilisé OneHotEncoder pour faire cette transformation automatiquement, avec drop="first" pour éviter la redondance entre les colonnes, ensuite fit_transform() pour apprendre les

catégories et transformer les données, puis j'ai créé un nouveau dataframe avec les colonnes encodées, j'ai supprimé les anciennes colonnes texte, et enfin j'ai ajouté les nouvelles colonnes numériques au dataset.

```python
categorical_cols = [
    "host_identity_verified",
    "instant_bookable",
    "cancellation_policy",
    "room type"
]

encoder = OneHotEncoder(drop="first", sparse_output=False)

encoded = encoder.fit_transform(data[categorical_cols])

encoded_df = pd.DataFrame(
    encoded,
    columns=encoder.get_feature_names_out(categorical_cols)
)

encoded_df.index = data.index

# Drop old columns
data = data.drop(columns=categorical_cols)

# Concat safely
data = pd.concat([data, encoded_df], axis=1)
```
✓ 0.0s

```python
data
```
✓ 0.0s Python

	Construction year	price	service fee	minimum nights	number of reviews	reviews per month	review rate number	calculated host listings count	host_identity_verified_verified	instant_bookable_True	cancellation_policy_moderate	cancellation_p
2	2005	620.0	124.000000	3	0.0	0.00	5.0	1.0	0.0	1.0	0.0	
5	2013	577.0	115.000000	3	74.0	0.59	3.0	1.0	1.0	1.0	0.0	
11	2008	606.0	121.000000	5	53.0	0.43	4.0	1.0	1.0	1.0	0.0	
19	2004	1095.0	125.039249	3	53.0	4.44	5.0	1.0	0.0	0.0	1.0	
20	2013	281.0	56.000000	7	0.0	0.00	3.0	2.0	1.0	0.0	1.0	
...	...	...	...	...	...	...	...	...	...	...	...	
102053	2012	696.0	125.039249	7	12.0	0.44	5.0	1.0	0.0	1.0	1.0	
102054	2012	909.0	125.039249	1	19.0	0.72	3.0	2.0	1.0	0.0	0.0	
102055	2012	387.0	125.039249	2	50.0	3.12	4.0	2.0	1.0	1.0	1.0	
102056	2012	848.0	125.039249	2	0.0	0.00	1.0	1.0	0.0	1.0	0.0	
102057	2011	1128.0	125.039249	4	22.0	0.85	4.0	1.0	1.0	0.0	0.0	

866 rows × 15 columns

2-J'ai appliqué la **normalisation (MinMaxScaler)** sur les colonnes numériques (**price**, **service fee**, **minimum nights**, **number of reviews**) pour transformer les valeurs entre **0 et 1**, parce que les données ont des échelles différentes (par exemple price peut être grand alors que number of reviews est petit), ce qui peut influencer le modèle.

Le principe du **MinMaxScaler** est simple : il prend la valeur minimale et maximale de chaque colonne, puis il transforme chaque valeur pour qu'elle soit comprise entre **0 (min)** et **1 (max)**.

J'ai utilisé scaler.fit_transform() pour apprendre les valeurs min/max et appliquer la transformation, puis j'ai remplacé les anciennes colonnes par les nouvelles valeurs normalisées dans une copie du dataset (**data_minmax**).

```python
# Normalisation et standardisation
numeric_cols = [
    "price",
    "service fee",
    "minimum nights",
    "number of reviews"
]

scaler = MinMaxScaler()

data_minmax = data.copy()
data_minmax[numeric_cols] = scaler.fit_transform(data[numeric_cols])
data_minmax
```

✓ 0.0s Pytho

	Construction year	price	service fee	minimum nights	number of reviews	reviews per month	review rate number	calculated host listings count	host_identity_verified_verified	instant_bookable_True	cancellation_policy_moderate	cancellatio
2	2005	0.495652	0.495652	0.4	0.000000	0.00	5.0	1.0	0.0	1.0	0.0	
5	2013	0.458261	0.456522	0.4	0.880952	0.59	3.0	1.0	1.0	1.0	0.0	
11	2008	0.483478	0.482609	0.6	0.630952	0.43	4.0	1.0	1.0	1.0	0.0	
19	2004	0.908696	0.500171	0.4	0.630952	4.44	5.0	1.0	0.0	0.0	1.0	
20	2013	0.200870	0.200000	0.8	0.000000	0.00	3.0	2.0	1.0	0.0	1.0	

Partie 4 : Gestion des Données Déséquilibrées

Je n'ai pas travaillé sur cette partie parce que la colonne cible **price** n'est pas une variable catégorielle mais une variable numérique (continue).

La gestion des données déséquilibrées s'applique seulement aux problèmes de **classification**, où la variable cible contient des classes (par exemple : oui/non, 0/1), et où une classe est plus fréquente que l'autre.

Dans mon cas, **price** représente une valeur numérique (régression), donc il n'y a pas de classes à équilibrer, c'est pour cela que les méthodes comme **undersampling**, **oversampling** ou **SMOTE** ne sont pas adaptées ici.